

CS112: Wednesday 2 September, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM COURSE-MAP.YML AND THE TEACHING LOG

Monday we compiled. Source to object file to executable, and the difference between an error the compiler catches in one file and an error that only shows up when the files are joined.

Today: a variable is a fixed number of bytes sitting at an address. In Python a name points at an object and the size is none of your business. In C++ you choose the size when you declare the variable, it never changes, and whatever was in that memory before is still there until you put something in it.

TIME	BEAT
12:20	1. Types have sizes
12:32	2. Declaration is not initialization
12:45	3. The same number in three bases
13:00	4. Streams, and a first look at a pointer

Beat 1. Types have sizes (12 min)

```
cout << sizeof(char)    << endl;    // 1
cout << sizeof(int)     << endl;    // 4
cout << sizeof(double) << endl;    // 8
```

Talk to your neighbor. Why would a language make you choose?

Because the compiler has to lay the variable out before the program runs, and because four bytes of arithmetic is faster than eight when four is enough. Then `const`, which is the other half of the same idea: a promise about a value that the compiler holds you to.

Beat 2. Declaration is not initialization (13 min)

```
int x;
cout << x << endl;           // whatever was in that memory
```

Run it. Run it again. Run it after running something else. The number changes, or worse, it does not, and the bug survives until it matters.

Declaring a variable reserves space. It does not put anything in it.

Python has no equivalent of this, which is exactly why it catches people from Python.

Beat 3. The same number in three bases (15 min)

```
int n = 255;
cout << hex << n << endl;      // ff
cout << oct << n << endl;       // 377
cout << dec << n << endl;       // 255
```

One value, three spellings. `0x` and `0b` prefixes write them directly.

This is not trivia. Addresses print in hexadecimal, and on Friday every pointer you look at will be a hex number. Two hex digits are one byte, which is the reason anyone uses it.

Beat 4. Streams, and a first look at a pointer (13 min)

`cout`, `cin`, `cerr`, and an `ifstream` reading from a file. All the same shape: a stream is a thing you push values into or pull them out of, and the console is not special.

Then the first pointer:

```
int x = 7;
cout << &x << endl;      // the ADDRESS of x, in hex
```

What is that number?

Friday is the whole answer.